EcoBudget

Document Technique de Synthèse — Migration Kotlin Multiplatform

Projet 2025–2026 • Cibles Android et iOS • Module partagé **shared**

2. Architecture cible

```
EcoBudget
■
■■■ androidApp
■ ■■■ MainActivity.kt
■
■■■ shared
■ ■■■ commonMain
■ ■ ■■■ model
■ ■ ■■■ data/repository
■ ■ ■■■ viewmodel
■ ■ ■■■ ui/components
■ ■ ■■■ ui/screens
■ ■ ■■■ ui/theme
■ ■ ■■■ CommonUtils.kt
■ ■ ■■■ Strings.kt
■ ■ ■■■ App.kt
■ ■
■ ■■■ iosMain
■ ■■■ MainViewController.kt
■
■■■ iosApp
■■■ hôte SwiftUI
```

Le principe retenu est de placer dans `commonMain` tout ce qui ne dépend pas du système d'exploitation : modèles, règles de calcul, état de l'application, dépôt en mémoire, composants Compose et écran principal.

Les points d'entrée restent spécifiques aux plateformes :

• Android : `MainActivity.kt`

• iOS : `MainViewController.kt` et l'hôte SwiftUI.

---

3. Mise en place de l'infrastructure partagée

3.1 Module `shared`

Le module partagé est configuré comme bibliothèque Kotlin Multiplatform :

```
plugins {
kotlin("multiplatform")
id("com.android.library")
id("org.jetbrains.compose")
id("org.jetbrains.kotlin.plugin.compose")
}
```

Les cibles configurées sont :

• `androidTarget()`

• `iosX64()`

• `iosArm64()`

• `iosSimulatorArm64()`

Les cibles iOS produisent également un framework statique nommé `Shared`, ce qui permet son intégration dans l'application iOS.

3.2 Dépendances communes

Le module `commonMain` utilise notamment :

• Compose Runtime ;

• Compose Foundation ;

• Compose Material 3 ;

• Compose Material Icons ;

• `kotlinx-coroutines-core` ;

• `kotlinx-datetime`.

Ces bibliothèques sont adaptées à l'utilisation multiplateforme.

3.3 Liaison avec Android

`androidApp` dépend directement de :

```
implementation(project(":shared"))
```

L'application Android conserve donc uniquement son rôle d'hôte natif tandis que l'interface et la logique EcoBudget résident dans `shared`.

---

4. Migration de la couche Domaine

4.1 `Category.kt`

Problème rencontré

Le modèle utilisait initialement une référence vers `com.example.R`, ce qui reproduisait le fonctionnement des ressources Android dans le domaine.

Solution

Le modèle utilise maintenant des valeurs métier simples :

```
enum class Category(
val label: String,
val emoji: String
)
```

Les catégories sont :

• Transport ;

• Alimentation ;

• Loisirs ;

• Logement.

Justification

Le domaine ne connaît plus le mécanisme de ressources Android. Il ne dépend que de types Kotlin standards. Il peut donc être compilé pour Android et iOS.

4.2 `Transaction.kt`

Problème rencontré

La transaction doit pouvoir être partagée sans dépendre d'une classe Android.

Solution

Le modèle est une `data class` Kotlin pure :

```
data class Transaction(
val id: String,
val title: String,
val amount: Double,
val date: Long,
val category: Category
)
```

Justification

`String`, `Double`, `Long`, `data class` et l'énumération `Category` sont indépendants de la plateforme.

4.3 `YearMonth.kt`

Problème rencontré

La navigation mensuelle et la détermination du mois d'une transaction ne doivent pas dépendre de `java.time` ou `java.util.Calendar`.

Solution

La gestion temporelle est réalisée avec `kotlinx-datetime` :

• `Clock.System.now()`

• `Instant`

• `TimeZone`

• `LocalDateTime`

• `LocalDate`

Une méthode `plusMonths()` a également été ajoutée pour construire les données de démonstration sur les mois précédent et suivant.

Justification

`kotlinx-datetime` fournit une API commune à Kotlin Multiplatform et évite les classes Java spécifiques à Android/JVM.

---

5. Migration de la couche Données

5.1 `TransactionRepository.kt`

Problème rencontré

Le contrat du repository devait pouvoir être consommé par la logique commune sans connaître une technologie de stockage Android.

Solution

L'interface expose un `Flow` et des opérations suspendues :

```
interface TransactionRepository {
fun getTransactions(): Flow<List<Transaction>>
suspend fun addTransaction(transaction: Transaction)
suspend fun updateTransaction(transaction: Transaction)
```

```
suspend fun deleteTransaction(id: String)
}
```

Justification

`Flow` et les fonctions `suspend` appartiennent à Kotlin Coroutines et sont utilisables dans une architecture KMP.

5.2 `FakeTransactionRepository.kt`

Problèmes rencontrés

Le repository de démonstration utilisait :

```
java.util.Calendar
java.util.UUID
```

Ces imports sont incompatibles avec `commonMain`.

Solution

• `Calendar` a été remplacé par `YearMonth` + `kotlinx-datetime`.

• La génération d'identifiant utilise une combinaison d'horodatage et de valeur aléatoire Kotlin.

• Les transactions restent stockées dans un `MutableStateFlow`.

Justification

La donnée de test reste entièrement en mémoire et ne dépend d'aucune API JVM. Le même repository peut donc être utilisé dans les deux plateformes.

---

6. Migration de la couche Présentation

6.1 `EcoBudgetViewModel.kt`

Problème rencontré

Un `androidx.lifecycle.ViewModel` ne peut pas être placé tel quel dans `commonMain`.

Solution

Le ViewModel métier a été transformé en classe Kotlin commune qui possède :

• un `CoroutineScope` ;

• des `MutableStateFlow` privés ;

• un `StateFlow<EcoBudgetUiState>` public ;

• une méthode `clear()` appelée à la destruction du composant racine.

La logique réactive utilise :

```
combine(...)
stateIn(...)
SharingStarted.WhileSubscribed(...)
```

Le scope partagé utilise `Dispatchers.Default` afin de ne pas imposer le dispatcher Android `Dispatchers.Main` au code commun.

Justification

La logique de gestion d'état est ainsi indépendante d'Android. Le cycle de vie de l'objet est contrôlé par `App.kt`, ce qui évite de dépendre de `androidx.lifecycle.ViewModel`.

6.2 `EcoBudgetUiState`

Cette classe centralise l'état nécessaire à l'écran :

• mois courant ;

• transactions du mois ;

• transactions filtrées ;

• catégories sélectionnées ;

• budget mensuel ;

• total dépensé ;

• montant restant ;

• état du dialogue ;

• transaction en cours d'édition.

Les propriétés calculées `budgetUsageRatio`, `budgetUsagePercentage` et `isAllCategoriesSelected` restent entièrement déterministes et multiplateformes.

---

7. Migration de l'interface Compose

Les fichiers suivants sont placés dans `commonMain` :

Fichier	Responsabilité	Traitement KMP
`App.kt`	Racine de l'application	Compose partagé + gestion du scope
`EcoBudgetScreen.kt`	Tableau de bord	UI partagée
`MonthNavigatorBar.kt`	Navigation mensuelle	UI partagée
`AddTransactionDialog.kt`	Ajout/modification	UI partagée
`TransactionCard.kt`	Affichage d'une dépense	UI partagée
`Color.kt`	Palette	API Compose commune
`Theme.kt`	Thème Material	API Compose commune
`Type.kt`	Typographie	API Compose commune

---

8. `TransactionCard.kt` : suppression des API Java

Problème rencontré

Le composant utilisait :

```
java.text.NumberFormat
java.text.SimpleDateFormat
java.util.Date
java.util.Locale
```

Ces classes ne doivent pas apparaître dans `commonMain`.

Solution

Le formatage est centralisé dans `CommonUtils.kt`.

Le montant est formaté par :

```
formatFcfa(transaction.amount)
```

La date relative utilise :

```
formatRelativeDate(...)
```

Cette fonction s'appuie elle-même sur `kotlinx-datetime`.

Justification

Le formatage des montants et des dates ne dépend plus du runtime Java. L'interface est donc portable vers iOS.

---

9. Centralisation des ressources textuelles

Problème rencontré

Le projet Android utilisait normalement le mécanisme :

```
R.string.*
androidx.compose.ui.res.stringResource
```

Ce mécanisme n'est pas directement disponible comme tel dans le code commun.

Solution appliquée

Un catalogue partagé est placé dans :

```
shared/src/commonMain/kotlin/com/example/Strings.kt
```

Il expose :

```
object SharedStrings
```

et une fonction commune :

```
fun stringResource(id: Int, vararg args: Any): String
```

Les textes de l'application sont donc centralisés dans `commonMain`.

Exemples :

• nom de l'application ;

• boutons ;

• navigation mensuelle ;

• catégories ;

• messages d'état vide ;

• libellés des montants ;

• textes d'accessibilité.

Justification

L'interface n'accède plus aux ressources Android. Les libellés peuvent être consommés par Android et iOS à partir du module partagé.

Évolution possible : pour une internationalisation complète, ce catalogue peut être remplacé ultérieurement par le système officiel Compose Multiplatform Resources avec des fichiers `strings.xml` partagés.

---

10. Gestion des dates et heures

Le projet utilise maintenant `kotlinx-datetime`.

La conversion d'un mois vers un timestamp est centralisée :

```
timestampForMonth(
year,
month,
day,
hour
)
```

La date est ensuite interprétée avec :

```
TimeZone.currentSystemDefault()
```

Cette approche évite les différences d'API entre JVM et Native.

---

11. Liste complète des fichiers migrés vers `commonMain`

Domaine

1. `model/Category.kt` 2. `model/Transaction.kt` 3. `model/YearMonth.kt`

Données

4. `data/repository/TransactionRepository.kt` 5. `data/repository/FakeTransactionRepository.kt`

Logique et état

6. `viewmodel/EcoBudgetViewModel.kt`

Utilitaires et ressources

7. `CommonUtils.kt` 8. `Strings.kt`

Présentation

9. `App.kt` 10. `ui/screens/EcoBudgetScreen.kt` 11. `ui/components/AddTransactionDialog.kt` 12. `ui/components/MonthNavigatorBar.kt` 13. `ui/components/TransactionCard.kt` 14. `ui/theme/Color.kt` 15. `ui/theme/Theme.kt` 16. `ui/theme/Type.kt`

Cible iOS

17. `iosMain/kotlin/com/example/MainViewController.kt`

---

12. Contrôle de neutralité de `commonMain`

Une recherche statique a été effectuée sur le code commun pour détecter les imports suivants :

```
java.*
javax.*
android.*
androidx.lifecycle.ViewModel
androidx.compose.ui.res.stringResource
```

Après correction, aucun de ces imports n'est présent dans `shared/src/commonMain`.

Cette vérification est particulièrement importante car elle permet de détecter rapidement les dépendances JVM/Android qui empêcheraient la compilation Native.

---

13. Validation fonctionnelle attendue

Les scénarios suivants doivent être exécutés sur l'émulateur Android :

13.1 Tableau de bord

Vérifier :

• affichage du nom EcoBudget ;

• affichage du budget restant ;

• affichage du total dépensé ;

• affichage du pourcentage consommé.

13.2 Navigation mensuelle

Tester :

1. mois courant ; 2. bouton précédent ; 3. bouton suivant ; 4. retour au mois courant.

Les données de démonstration contiennent des transactions sur le mois précédent, le mois courant et le mois suivant.

13.3 Filtres

Tester :

• `Tous` ;

• Transport ;

• Alimentation ;

• Loisirs ;

• Logement ;

• sélection de plusieurs catégories ;

• désélection ;

• retour à `Tous`.

13.4 Gestion des dépenses

Tester :

• ajout ;

• modification ;

• suppression ;

• validation d'un montant positif ;

• affichage immédiat de la transaction.

---

14. Procédure de validation Android

Depuis la racine du projet :

```
./gradlew :shared:compileKotlinAndroid
./gradlew :androidApp:assembleDebug
```

Puis installer l'APK généré sur un émulateur :

```
adb install -r androidApp/build/outputs/apk/debug/androidApp-debug.apk
```

Ou lancer directement :

```
./gradlew :androidApp:installDebug
```

Résultat dans l'environnement de génération

La commande de compilation a été lancée, mais le wrapper a échoué avant la compilation car il devait télécharger :

```
https://services.gradle.org/distributions/gradle-8.12-bin.zip
```

Erreur observée :

```
java.net.UnknownHostException: services.gradle.org
```

Il s'agit d'une limitation réseau de l'environnement de génération, et non d'un résultat de compilation permettant de conclure que le projet est compilable ou non.

---

15. Procédure iOS

Sur macOS avec Xcode :

1. ouvrir le projet iOS ; 2. configurer le framework `Shared` généré par le module KMP ; 3. lancer la cible iOS Simulator ; 4. vérifier que `MainViewController()` est bien appelé depuis l'hôte SwiftUI.

La cible KMP configure :

```
iosX64
iosArm64
iosSimulatorArm64
```

et produit le framework :

```
Shared
```

---

16. Historique Git proposé

L'historique préparé dans le projet suit les étapes de migration demandées :

```
de14e50 chore: initialise le projet KMP Android et iOS
489df84 feat: mettre en place le module shared et les cibles multiplateformes
4577fea refactor: migrer les modeles et la couche domaine vers commonMain
6e768a6 refactor: porter le repository et le ViewModel en logique partagee
835a91f refactor: centraliser les ressources textuelles partagees
9e41861 fix: supprimer les dependances Java et Android de commonMain
4f5d878 docs: ajouter la synthese technique et le guide de validation
```

Avant remise, l'étudiant doit remplacer l'identité Git locale par son identité personnelle et pousser le dépôt vers son GitHub.

---

17. Publication GitHub

Commandes recommandées :

```
git config user.name "Prénom Nom"
git config user.email "votre-email@example.com"

git remote add origin https://github.com/VOTRE_COMPTE/EcoBudget-KMP.git
git branch -M main
git push -u origin main
```

Le lien à placer dans le livrable est ensuite :

```
https://github.com/VOTRE_COMPTE/EcoBudget-KMP
```

---

18. Correspondance avec les critères d'évaluation

Critère	Éléments apportés
Configuration & structure — 4 pts	Module `shared`, cibles Android/iOS, framework `Shared`, dépendance `androidApp -> shared`
Domaine & logique — 4 pts	Modèles Kotlin purs, suppression de `java.*`/`android.*`, `kotlinx-datetime`
État & cycle de vie — 4 pts	`StateFlow`, `combine`, `stateIn`, scope commun, `clear()`
Validation Android — 3 pts	Commandes de build et scénarios fonctionnels documentés ; validation locale encore à exécuter
Document technique — 5 pts	Analyse fichier par fichier, problèmes, solutions et justifications

---

19. Conclusion

La migration transforme EcoBudget en une application structurée autour d'un module partagé KMP.

Le domaine, le repository de démonstration, la gestion d'état, les utilitaires, les ressources textuelles et l'interface Compose sont désormais regroupés dans `commonMain`.

Les principales dépendances Android/JVM identifiées lors de la migration — `Calendar`, `UUID`, `NumberFormat`, `SimpleDateFormat`, `Date`, `Locale`, `R.string` et `androidx.lifecycle.ViewModel` — ont été retirées du code commun.

La dernière étape à effectuer sur le poste de développement est la compilation Android réelle, l'installation sur émulateur et l'exécution des scénarios fonctionnels listés dans ce rapport. Cette étape est indispensable avant de déclarer la validation fonctionnelle comme acquise.